Backend Improvements Guide

This document outlines how to transform the backend from a basic implementation to a production-ready, professional service.

Overview

Current state: Functional Node.js/TypeScript API with critical security issues and missing DevOps practices.
Target state: Production-ready service with tests, CI/CD, and professional deployment practices.

TIER 1: CRITICAL FIXES (Do First)

1. Remove Hardcoded Credentials

Status: CRITICAL SECURITY ISSUE

File: `api/login.ts` line 16

```
// BEFORE (VULNERABLE)
const correctPassword = 'R3load@24680';

// AFTER
const correctPassword = process.env.VITE_PASSWORD || 'dev-password';
```

Steps:

1. Open `.env.local` and add:

   ```
   VITE_PASSWORD=your_actual_password
   OPENAI_API_KEY=sk-...
   ELEVENLABS_API_KEY=...
   VERCEL_URL=https://your-domain.com
   ```

2. Update `api/login.ts`:

   ```
   export default async function handler(req: NextApiRequest, res:
   NextApiResponse) {
     if (req.method === 'OPTIONS') {
       res.setHeader('Access-Control-Allow-Origin', '*');
       res.setHeader('Access-Control-Allow-Methods', 'GET, POST, OPTIONS');
       res.setHeader('Access-Control-Allow-Headers', 'Content-Type');
       return res.status(200).end();
     }

     const { password } = req.body;
     const correctPassword = process.env.VITE_PASSWORD;

     if (!correctPassword) {
       return res.status(500).json({ error: 'Server misconfigured' });
     }
   ```

```
  if (password !== correctPassword) {
    return res.status(401).json({ error: 'Invalid password' });
  }

  res.setHeader('Access-Control-Allow-Origin', '*');
  return res.status(200).json({ token: 'ok' });
}
```

3. Update `dev.ts` to use environment variable:

```
const password = process.env.VITE_PASSWORD || 'dev-password';
```

4. Add `.gitignore`:

```
.env.local
.env*.local
*.swp
.DS_Store
node_modules/
dist/
.next/
```

---

2. Fix Windows Compatibility

Status: HIGH - Blocks Windows developers

Files: `api/transcribe.ts`, `api/dev.ts`

```
// BEFORE
const tempFile = `/tmp/audio-${Date.now()}.wav`;

// AFTER
import { tmpdir } from 'os';
import { join } from 'path';

const tempDir = tmpdir();
const tempFile = join(tempDir, `audio-${Date.now()}.wav`);
```

In `api/transcribe.ts`:

```
import { writeFileSync, unlinkSync } from 'fs';
import { tmpdir } from 'os';
import { join } from 'path';
import FormData from 'form-data';
import fs from 'fs';

export default async function handler(req: NextApiRequest, res: NextApiResponse) {
  // ... CORS and method checks ...
```

```
  try {
    const buffer = Buffer.from(req.body.audio, 'base64');
    const tempFile = join(tmpdir(), `audio-${Date.now()}.wav`);

    writeFileSync(tempFile, buffer);

    try {
      const formData = new FormData();
      formData.append('file', fs.createReadStream(tempFile));
      formData.append('model', 'whisper-1');

      const response = await fetch('https://api.openai.com/v1/audio/transcriptions',
{
        method: 'POST',
        headers: {
          ...formData.getHeaders(),
          Authorization: `Bearer ${process.env.OPENAI_API_KEY}`,
        },
        body: formData,
      });

      if (!response.ok) {
        throw new Error(`Whisper API error: ${response.statusText}`);
      }

      const data = await response.json();
      res.status(200).json({ text: data.text });
    } finally {
      // Guaranteed cleanup
      try {
        unlinkSync(tempFile);
      } catch (err) {
        console.error(`Failed to clean up temp file: ${tempFile}`, err);
      }
    }
  } catch (error) {
    console.error('Transcription error:', error);
    res.status(500).json({
      error: error instanceof Error ? error.message : 'Transcription failed'
    });
  }
}
```

In `api/dev.ts`:

```
import { tmpdir } from 'os';
import { join } from 'path';

// Replace all `/tmp/` references with:
const tempFile = join(tmpdir(), `audio-${Date.now()}.wav`);
```

---

3. Fix Unused Dependencies

File: `package.json`

```
# Remove unused packages
npm uninstall axios @supabase/supabase-js

# Verify no imports reference these
grep -r "axios\|supabase" api/
grep -r "axios\|supabase" client/src/
```

---

4. Add Type Definitions

Create `api/types.ts`:

```
export interface Message {
  role: 'user' | 'assistant';
  content: string;
}

export interface ChatRequest {
  message: string;
  sessionId?: string;
}

export interface ChatResponse {
  reply: string;
}

export interface TranscribeRequest {
  audio: string; // base64
}

export interface TranscribeResponse {
  text: string;
}

export interface TTSRequest {
  text: string;
}

export interface TTSResponse {
  audio: string; // base64 or URL
}

export interface SummarizeRequest {
  messages: Message[];
}

export interface SummarizeResponse {
  transcript: string;
```

```
  summary: string;
}

export interface TakeawaysRequest {
  messages: Message[];
}

export interface TakeawaysResponse {
  takeaways: string[];
}

export interface ErrorResponse {
  error: string;
}
```

Then use in handlers:

```
// Before
const response = await fetch('...');

// After
import type { ChatResponse, ErrorResponse } from './types';

const response = await fetch('...');
const data: ChatResponse | ErrorResponse = await response.json();
```

TIER 2: HIGH-IMPACT IMPROVEMENTS (Week 1)

5. Extract CORS Middleware

Create `api/_cors.ts`:

```
import { NextApiRequest, NextApiResponse } from 'next';

export function setCorsHeaders(res: NextApiResponse): void {
  res.setHeader('Access-Control-Allow-Origin', process.env.VITE_API_URL || '*');
  res.setHeader('Access-Control-Allow-Methods', 'GET, POST, OPTIONS');
  res.setHeader('Access-Control-Allow-Headers', 'Content-Type, Authorization');
  res.setHeader('Access-Control-Max-Age', '86400');
}

export function handleCors(req: NextApiRequest, res: NextApiResponse): boolean {
  setCorsHeaders(res);

  if (req.method === 'OPTIONS') {
    res.status(200).end();
    return true;
  }
```

```
  return false;
}
```

Update all handlers:

```
// Before
if (req.method === 'OPTIONS') {
  res.setHeader('Access-Control-Allow-Origin', '*');
  res.setHeader('Access-Control-Allow-Methods', 'GET, POST, OPTIONS');
  res.setHeader('Access-Control-Allow-Headers', 'Content-Type');
  return res.status(200).end();
}

// After
import { handleCors } from './_cors';

if (handleCors(req, res)) return;
```

---

6. Add ESLint Configuration

Create `.eslintrc.cjs`:

```
module.exports = {
  root: true,
  env: { node: true, es2021: true },
  extends: [
    'eslint:recommended',
    'plugin:@typescript-eslint/recommended',
  ],
  parser: '@typescript-eslint/parser',
  parserOptions: {
    ecmaVersion: 2021,
    sourceType: 'module',
  },
  plugins: ['@typescript-eslint'],
  rules: {
    '@typescript-eslint/no-explicit-any': 'error',
    '@typescript-eslint/no-unused-vars': ['error', {
      argsIgnorePattern: '^_',
      varsIgnorePattern: '^_'
    }],
    '@typescript-eslint/explicit-module-boundary-types': 'off',
    'no-console': ['warn', { allow: ['warn', 'error'] }],
    'prefer-const': 'error',
    'eqeqeq': ['error', 'always'],
  },
};
```

Update `package.json`:

```
{
  "scripts": {
    "lint": "eslint api/ --ext .ts,.tsx",
    "lint:fix": "eslint api/ --ext .ts,.tsx --fix"
  },
  "devDependencies": {
    "eslint": "^8.0.0",
    "@typescript-eslint/eslint-plugin": "^6.0.0",
    "@typescript-eslint/parser": "^6.0.0"
  }
}
```

Run:

```
npm install
npm run lint:fix
```

---

7. Add Test Suite

Create `tests/api.test.ts`:

```
import { describe, it, expect, beforeAll, afterAll } from 'vitest';

const API_URL = 'http://localhost:3001';

describe('API Endpoints', () => {
  // Authentication
  describe('POST /api/login', () => {
    it('should reject invalid password', async () => {
      const response = await fetch(`${API_URL}/api/login`, {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ password: 'wrong' }),
      });
      expect(response.status).toBe(401);
    });

    it('should accept valid password', async () => {
      const response = await fetch(`${API_URL}/api/login`, {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ password: process.env.VITE_PASSWORD }),
      });
      expect(response.status).toBe(200);
      const data = await response.json();
      expect(data.token).toBeDefined();
    });
  });

  // Chat API
```

```
describe('POST /api/chat', () => {
  it('should reject request without message', async () => {
    const response = await fetch(`${API_URL}/api/chat`, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({}),
    });
    expect(response.status).toBe(400);
  });

  it('should return a coaching response', async () => {
    const response = await fetch(`${API_URL}/api/chat`, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({
        message: 'Hello, I want to practice English'
      }),
    });
    expect(response.status).toBe(200);
    const data = await response.json();
    expect(data.reply).toBeDefined();
    expect(typeof data.reply).toBe('string');
  });

  it('should handle empty message gracefully', async () => {
    const response = await fetch(`${API_URL}/api/chat`, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ message: '' }),
    });
    // Should either reject or handle gracefully
    expect([400, 200]).toContain(response.status);
  });
});

// CORS
describe('CORS Headers', () => {
  it('should set CORS headers on all responses', async () => {
    const response = await fetch(`${API_URL}/api/chat`, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ message: 'test' }),
    });
    expect(response.headers.get('access-control-allow-origin')).toBeDefined();
  });

  it('should handle OPTIONS requests', async () => {
    const response = await fetch(`${API_URL}/api/chat`, {
      method: 'OPTIONS',
    });
    expect(response.status).toBe(200);
  });
```

```
  });

  // Input validation
  describe('Input Validation', () => {
    it('should reject messages that are too long', async () => {
      const longMessage = 'a'.repeat(10000);
      const response = await fetch(`${API_URL}/api/chat`, {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ message: longMessage }),
      });
      // Should reject or truncate
      expect(response.status).toBeLessThan(500);
    });
  });
});
```

Update `package.json`:

```
{
  "scripts": {
    "test": "vitest run",
    "test:watch": "vitest --watch",
    "test:coverage": "vitest run --coverage"
  },
  "devDependencies": {
    "vitest": "^1.0.0",
    "@vitest/coverage-v8": "^1.0.0"
  }
}
```

---

8. Add CI/CD Pipeline

Create `.github/workflows/ci.yml`:

```
name: CI/CD Pipeline

on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]

jobs:
  test-and-lint:
    runs-on: ubuntu-latest

    strategy:
      matrix:
        node-version: [18.x, 20.x]
```

```
    steps:
      - uses: actions/checkout@v4

      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: ${{ matrix.node-version }}
          cache: 'npm'

      - name: Install dependencies
        run: npm ci

      - name: Run linter
        run: npm run lint

      - name: Run tests
        run: npm run test
        env:
          VITE_PASSWORD: test-password
          OPENAI_API_KEY: ${{ secrets.OPENAI_API_KEY }}
          ELEVENLABS_API_KEY: ${{ secrets.ELEVENLABS_API_KEY }}

      - name: Build
        run: npm run build

  security-check:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Run npm audit
        run: npm audit --audit-level=moderate
```

Create `.github/workflows/deploy.yml`:

```
name: Deploy to Vercel

on:
  push:
    branches: [main]

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Deploy to Vercel
        uses: vercel/action@v4
        with:
          vercel-token: ${{ secrets.VERCEL_TOKEN }}
```

```
          vercel-org-id: ${{ secrets.VERCEL_ORG_ID }}
          vercel-project-id: ${{ secrets.VERCEL_PROJECT_ID }}
```

---

9. Add Error Logging

Create `api/_logger.ts`:

```
export interface LogContext {
  endpoint: string;
  method: string;
  statusCode?: number;
  duration?: number;
  error?: Error;
}

export function logRequest(context: LogContext): void {
  const timestamp = new Date().toISOString();
  const { endpoint, method, statusCode, duration, error } = context;

  const level = error ? 'ERROR' : statusCode && statusCode >= 400 ? 'WARN' : 'INFO';
  const durationMs = duration ? `${duration}ms` : 'unknown';

  console.error(JSON.stringify({
    timestamp,
    level,
    endpoint,
    method,
    statusCode,
    duration: durationMs,
    error: error?.message || null,
  }));
}

export function logError(error: unknown, context: string): void {
  console.error(JSON.stringify({
    timestamp: new Date().toISOString(),
    level: 'ERROR',
    context,
    message: error instanceof Error ? error.message : String(error),
    stack: error instanceof Error ? error.stack : null,
  }));
}
```

Use in handlers:

```
import { logRequest, logError } from './_logger';

export default async function handler(req: NextApiRequest, res: NextApiResponse) {
  const startTime = Date.now();
```

```
  try {
    if (handleCors(req, res)) return;

    // Handler logic...

    logRequest({
      endpoint: '/api/chat',
      method: req.method || 'UNKNOWN',
      statusCode: 200,
      duration: Date.now() - startTime,
    });
  } catch (error) {
    logError(error, 'chat handler');
    res.status(500).json({ error: 'Internal server error' });
  }
}
```

TIER 3: PROFESSIONAL POLISH (Week 2)

10. Add CONTRIBUTING.md

Create `CONTRIBUTING.md`:

```
# Contributing to LangCoach

Thank you for your interest in contributing! This guide will help you get started.

## Getting Started

1. Clone the repository:
   \`\`\`bash
   git clone https://github.com/yourusername/langcoach.git
   cd langcoach
   \`\`\`

2. Install dependencies:
   \`\`\`bash
   npm install
   \`\`\`

3. Create `.env.local`:
   \`\`\`
   VITE_PASSWORD=your_password
   OPENAI_API_KEY=sk-...
   ELEVENLABS_API_KEY=...
   \`\`\`

4. Start development server:
   \`\`\`bash
   npm run dev
   \`\`\`
```

Code Standards

TypeScript
- No `any` types without justification
- Explicit return types on functions
- Interfaces for API contracts

API Handlers
- Add types for request/response
- Use CORS middleware
- Handle all error cases
- Log errors with context

Commits
Use conventional commit format:
- \`feat: add voice recording\`
- \`fix: handle transcription errors\`
- \`docs: update README\`
- \`test: add chat API tests\`

Testing
- Write tests for new endpoints
- Aim for >80% coverage
- Run \`npm run test\` before submitting PR

Linting
- Run \`npm run lint:fix\` before committing
- All checks must pass in CI

Pull Request Process

1. Create feature branch:
   \`\`\`bash
   git checkout -b feat/your-feature
   \`\`\`

2. Make changes and test:
   \`\`\`bash
   npm run lint
   npm run test
   \`\`\`

3. Commit with conventional format
4. Push and open PR against \`main\`
5. Wait for CI to pass and request review

Reporting Issues

- Use GitHub issues for bug reports
- Include: what you expected, what happened, steps to reproduce
- Add screenshots/error logs when relevant

```
## Questions?

Open an issue or reach out to maintainers.
```

---

11. Add CHANGELOG.md

Create `CHANGELOG.md`:

```
# Changelog

All notable changes to this project will be documented in this file.

The format is based on [Keep a Changelog](https://keepachangelog.com/en/1.0.0/),
and this project adheres to [Semantic Versioning]
(https://semver.org/spec/v2.0.0.html).

## [1.0.0] - 2026-08-10

### Added
- Initial release with voice-based English coaching
- Real-time speech-to-text using OpenAI Whisper API
- GPT-4 powered coaching responses with compassionate prompting
- Text-to-speech responses using ElevenLabs API
- User authentication with password protection
- Session management and conversation history
- Transcript generation and download
- Key takeaways extraction from sessions
- Responsive React UI with design system
- Mobile-first approach with 3-column desktop layout
- Dark mode CSS support
- Comprehensive error handling

### Security
- Environment variable credential management
- CORS protection on all API endpoints
- Input validation and sanitization
- Type-safe API contracts with TypeScript

### Infrastructure
- Vercel serverless deployment
- Next.js API routes
- Cross-platform compatible (Windows, Mac, Linux)
- Proper temp file cleanup

### Known Limitations
- Requires valid OpenAI API key (paid)
- Requires valid ElevenLabs API key (paid)
- Session data stored in browser only (no persistence)
- 60-second recording limit
- Conversations lost on page refresh
```

```
### Future Roadmap
- [ ] Database persistence for sessions
- [ ] User accounts and multi-device sync
- [ ] Conversation editing and message deletion
- [ ] Recording playback preview before send
- [ ] Waveform visualization during recording
- [ ] Metrics and progress tracking
- [ ] Export to PDF/DOCX
- [ ] Dark mode toggle UI
- [ ] Advanced coach customization
```

---

12. Add Input Validation

Create `api/_validation.ts`:

```
export interface ValidationError {
  field: string;
  message: string;
}

export function validateChatMessage(message: string): ValidationError | null {
  if (!message || typeof message !== 'string') {
    return { field: 'message', message: 'Message is required' };
  }

  if (message.trim().length === 0) {
    return { field: 'message', message: 'Message cannot be empty' };
  }

  if (message.length > 5000) {
    return { field: 'message', message: 'Message must be less than 5000 characters'
};
  }

  return null;
}

export function validatePassword(password: string): ValidationError | null {
  if (!password || typeof password !== 'string') {
    return { field: 'password', message: 'Password is required' };
  }

  if (password.length === 0) {
    return { field: 'password', message: 'Password cannot be empty' };
  }

  return null;
}

export function validateAudio(audioBase64: string): ValidationError | null {
```

```
  if (!audioBase64) {
    return { field: 'audio', message: 'Audio data is required' };
  }

  if (audioBase64.length > 10_000_000) { // ~10MB
    return { field: 'audio', message: 'Audio file too large' };
  }

  return null;
}
```

Use in handlers:

```
import { validateChatMessage } from './_validation';

export default async function handler(req: NextApiRequest, res: NextApiResponse) {
  if (handleCors(req, res)) return;

  const { message } = req.body;

  const error = validateChatMessage(message);
  if (error) {
    return res.status(400).json({ error: error.message });
  }

  // Handler logic...
}
```

---

13. Add Rate Limiting (Optional)

Create `api/_rateLimit.ts`:

```
const requestCounts = new Map<string, number[]>();
const WINDOW_MS = 60000; // 1 minute
const MAX_REQUESTS = 30;

export function isRateLimited(clientId: string): boolean {
  const now = Date.now();
  const requests = requestCounts.get(clientId) || [];

  // Remove old requests outside the window
  const recentRequests = requests.filter(time => now - time < WINDOW_MS);

  if (recentRequests.length >= MAX_REQUESTS) {
    return true;
  }

  // Add current request
  recentRequests.push(now);
  requestCounts.set(clientId, recentRequests);
```

```
  return false;
}
```

QUICK CHECKLIST

- [ ] Remove hardcoded password
- [ ] Fix Windows compatibility (tmpdir)
- [ ] Remove unused dependencies
- [ ] Add type definitions (types.ts)
- [ ] Extract CORS middleware
- [ ] Add ESLint config
- [ ] Add test suite with >80% coverage
- [ ] Add GitHub Actions CI/CD
- [ ] Add error logging
- [ ] Add CONTRIBUTING.md
- [ ] Add CHANGELOG.md
- [ ] Add input validation
- [ ] Add to .gitignore
- [ ] Verify all tests pass
- [ ] Run `npm run lint` with no errors

DEPLOYMENT

Once all improvements are complete:

```
# Verify everything works locally
npm run lint
npm run test
npm run build

# Push to GitHub
git add .
git commit -m "chore: add backend improvements and DevOps setup"
git push

# Monitor CI/CD in GitHub Actions
# Deploy to Vercel when CI passes
```

SECURITY CHECKLIST

- [ ] No hardcoded secrets in code
- [ ] Environment variables for all credentials
- [ ] CORS properly configured
- [ ] Input validation on all endpoints

- [ ] Error messages don't leak sensitive info
- [ ] Dependencies audited with `npm audit`
- [ ] No known vulnerabilities in dependencies

MONITORING & OBSERVABILITY

Once deployed to production, consider adding:

1. **Error Tracking:** Sentry

   ```
   import * as Sentry from "@sentry/nextjs";
   Sentry.init({ dsn: process.env.SENTRY_DSN });
   ```

2. **Performance Monitoring:** Vercel Analytics (built-in)
3. **Logging:** Vercel Logs or LogRocket
4. **Uptime Monitoring:** UptimeRobot or Pingdom

Questions? Check the main README.md or open an issue.